ALL 9 EXPERIMENTS

OpenMP and MPI – Complete Code Collection

Compiled from the nine experiment PDFs provided. Includes each program and the modifications shown in the extracted material.

Contents: Experiment 1 – Merge Sort; Experiment 2 – Static Scheduling and modifications; Experiment 3 – Fibonacci Tasks; Experiment 4 – Prime Numbers; Experiment 5 – MPI Send/Receive; Experiment 6 – Deadlock and Avoidance; Experiment 7 – Broadcast; Experiment 8 – Scatter/Gather; Experiment 9 – Reduce/Allreduce.

Experiment 1 – Parallel Merge Sort

Sequential and Parallel Merge Sort using OpenMP Tasks.

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define MIN_SIZE 10000

// Merge function
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int *L = (int *)malloc(n1 * sizeof(int));
    int *R = (int *)malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];

    free(L);
    free(R);
}

// Sequential Merge Sort
void mergeSortSequential(int arr[], int left, int right) {
```

```
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSortSequential(arr, left, mid);
        mergeSortSequential(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

// Parallel Merge Sort using tasks
void mergeSortParallel(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        if (right - left > MIN_SIZE) {
            #pragma omp task
            mergeSortParallel(arr, left, mid);

            #pragma omp task
            mergeSortParallel(arr, mid + 1, right);

            #pragma omp taskwait
        } else {
            mergeSortSequential(arr, left, mid);
            mergeSortSequential(arr, mid + 1, right);
        }

        merge(arr, left, mid, right);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int *arr1 = (int *)malloc(n * sizeof(int));
    int *arr2 = (int *)malloc(n * sizeof(int));

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr1[i]);
        arr2[i] = arr1[i];
    }

    double start, end;

    // Sequential sort
    start = omp_get_wtime();
    mergeSortSequential(arr1, 0, n - 1);
    end = omp_get_wtime();
    printf("Sequential Merge Sort Time: %f seconds\n", end - start);

    // Parallel sort
    start = omp_get_wtime();
    #pragma omp parallel
    {
        #pragma omp single
```

```
        mergeSortParallel(arr2, 0, n - 1);
    }
    end = omp_get_wtime();
    printf("Parallel Merge Sort Time: %f seconds\n", end - start);

    free(arr1);
    free(arr2);
    return 0;
}
```

Experiment 2 – OpenMP Static Scheduling

Program 1: Static scheduling with chunks of two iterations.

```
#include <stdio.h>
#include <omp.h>

int main() {
    int n;
    printf("Enter number of iterations: ");
    scanf("%d", &n);

    omp_set_num_threads(4);

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();

        #pragma omp for schedule(static, 2)
        for (int i = 0; i < n; i++) {
            printf("Thread %d executes iteration %d\n", tid, i);
        }
    }
    return 0;
}
```

Experiment 2 – Modification 1

Threads computing squares of numbers.

```
#include <stdio.h>
#include <omp.h>

int main() {
    int n;
    printf("Enter number of iterations: ");
    scanf("%d", &n);

    omp_set_num_threads(4);

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();

        #pragma omp for schedule(static, 2)
```

```
        for (int i = 0; i < n; i++) {
            int result = i * i;
            printf("Thread %d computed square of %d = %d\n",
                   tid, i, result);
        }
    }
    return 0;
}
```

Experiment 2 – Modification 2

Threads performing matrix multiplication using static scheduling.

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 6

int main() {
    int A[N][N], B[N][N], C[N][N];

    // Initialize matrices A and B
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            A[i][j] = i + j;
            B[i][j] = (i == j) ? 1 : 0;
            C[i][j] = 0;
        }
    }

    // Parallel multiplication
    #pragma omp parallel for schedule(static, 2)
    for (int i = 0; i < N; i++) {
        int tid = omp_get_thread_num();
        for (int j = 0; j < N; j++) {
            for (int k = 0; k < N; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
        printf("Thread %d computed row %d\n", tid, i);
    }

    printf("\nResult Matrix C:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%4d", C[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Experiment 3 – Fibonacci Using OpenMP Tasks

Calculate the nth Fibonacci number using OpenMP tasks.

```
#include <stdio.h>
#include <omp.h>

int fib(int n)
{
    int i, j;

    if (n < 2)
        return n;
    else
    {
        #pragma omp task shared(i) firstprivate(n)
        i = fib(n - 1);

        #pragma omp task shared(j) firstprivate(n)
        j = fib(n - 2);

        #pragma omp taskwait
        return i + j;
    }
}

int main()
{
    int n = 10;

    omp_set_dynamic(0);
    omp_set_num_threads(4);

    #pragma omp parallel shared(n)
    {
        #pragma omp single
        printf("fib(%d) = %d\n", n, fib(n));
    }

    return 0;
}
```

Experiment 4 – Prime Numbers Using OpenMP

Find prime numbers from 1 to n using the parallel for directive.

```
#include <stdio.h>
#include <omp.h>

void main()
{
    int prime[1000], i, j, n;

    // Prompt user for input
    printf("\n In order to find prime numbers from 1 to n, enter the value of n:");
    scanf("%d", &n);

    // Initialize all numbers as prime
    for (i = 1; i <= n; i++) {
```

```
        prime[i] = 1;
    }

    // 1 is not a prime number
    prime[1] = 0;

    // Sieve of Eratosthenes with parallelization
    for (i = 2; i * i <= n; i++) {
        #pragma omp parallel for
        for (j = i * i; j <= n; j = j + i) {
            if (prime[j] == 1) {
                prime[j] = 0;
            }
        }
    }

    // Print prime numbers
    printf("\n Prime numbers from 1 to %d are\n", n);

    for (i = 2; i <= n; i++) {
        if (prime[i] == 1) {
            printf("%d\t", i);
        }
    }
    printf("\n");
}
```

Experiment 5 – MPI Send and Receive

Demonstration of MPI_Send() and MPI_Recv().

```
#include <stdio.h>
#include <string.h>
#include <mpi.h>

const int MAX_STRING = 100;

int main(void) {
    char greeting[MAX_STRING];
    int comm_sz;
    int my_rank;

    // Start up MPI
    MPI_Init(NULL, NULL);

    // Get the number of processes
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

    // Get my rank among all the processes
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    if (my_rank != 0) {
        // Create message
        sprintf(greeting, "Greetings from process %d of %d!",
                my_rank, comm_sz);
```

```
        // Send message to process 0
        MPI_Send(greeting, strlen(greeting) + 1, MPI_CHAR,
                 0, 0, MPI_COMM_WORLD);
    } else {
        // Print my message
        printf("Greetings from process %d of %d!\n",
               my_rank, comm_sz);

        for (int q = 1; q < comm_sz; q++) {
            // Receive message from process q
            MPI_Recv(greeting, MAX_STRING, MPI_CHAR, q,
                     0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

            // Print message from process q
            printf("%s\n", greeting);
        }
    }

    // Shut down MPI
    MPI_Finalize();
    return 0;
}
```

Experiment 6 – Deadlock (deadlock.c)

Demonstration of blocking send operations that can cause deadlock with two processes.

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define N 10000000

int main(int argc, char *argv[])
{
    int rank, size;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2) {
        if (rank == 0)
            printf("Run this program with 2 processes only!\n");
        MPI_Finalize();
        return 0;
    }

    // Allocate large message buffers
    int *send_data = (int *)malloc(N * sizeof(int));
    int *recv_data = (int *)malloc(N * sizeof(int));

    // Initialize data
    for (int i = 0; i < N; i++)
```

```
        send_data[i] = rank;

    if (rank == 0) {
        printf("Process 0: Sending large data to Process 1...\n");
        MPI_Send(send_data, N, MPI_INT, 1, 0, MPI_COMM_WORLD);

        printf("Process 0: Receiving data from Process 1...\n");
        MPI_Recv(recv_data, N, MPI_INT, 1, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    } else if (rank == 1) {
        printf("Process 1: Sending large data to Process 0...\n");
        MPI_Send(send_data, N, MPI_INT, 0, 0, MPI_COMM_WORLD);

        printf("Process 1: Receiving data from Process 0...\n");
        MPI_Recv(recv_data, N, MPI_INT, 0, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }

    printf("Process %d completed communication.\n", rank);

    free(send_data);
    free(recv_data);
    MPI_Finalize();
    return 0;
}
```

Experiment 6 – Avoiding Deadlock (avoid_deadlock.c)

Demonstration of receiving before sending on process 1 to avoid the send-send deadlock pattern.

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define N 10000000

int main(int argc, char *argv[])
{
    int rank, size;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2) {
        if (rank == 0)
            printf("Run this program with 2 processes only!\n");
        MPI_Finalize();
        return 0;
    }

    // Allocate large message buffers
    int *send_data = (int *)malloc(N * sizeof(int));
    int *recv_data = (int *)malloc(N * sizeof(int));

    // Initialize data
```

```
    for (int i = 0; i < N; i++)
        send_data[i] = rank;

    if (rank == 0) {
        printf("Process 0: Sending large data to Process 1...\n");
        MPI_Send(send_data, N, MPI_INT, 1, 0, MPI_COMM_WORLD);

        printf("Process 0: Receiving data from Process 1...\n");
        MPI_Recv(recv_data, N, MPI_INT, 1, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    } else if (rank == 1) {
        printf("Process 1: Receiving data from Process 0...\n");
        MPI_Recv(recv_data, N, MPI_INT, 0, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        printf("Process 1: Sending data to Process 0...\n");
        MPI_Send(send_data, N, MPI_INT, 0, 0, MPI_COMM_WORLD);
    }

    printf("Process %d: Communication successful! Received data[0] = %d\n",
           rank, recv_data[0]);

    free(send_data);
    free(recv_data);
    MPI_Finalize();
    return 0;
}
```

Experiment 7 – MPI Broadcast (Program 1)

Basic demonstration of MPI_Bcast().

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv)
{
    int rank, size;
    int data;

    // Initialize MPI
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Root process sets the data
    if (rank == 0) {
        data = 42;
        printf("Process %d is broadcasting data = %d\n",
               rank, data);
    }

    // Broadcast data from process 0 to all other processes
    MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

```
    // All processes print the received data
    printf("Process %d received data = %d\n", rank, data);

    MPI_Finalize();
    return 0;
}
```

Experiment 7 – MPI Broadcast (Program 2)

Broadcast with before-and-after messages.

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>

int main(int argc, char** argv)
{
    int answer = 0;
    int numProcs = 0, myRank = 0;

    MPI_Init(&argc, &argv);

    MPI_Comm_size(MPI_COMM_WORLD, &numProcs);
    MPI_Comm_rank(MPI_COMM_WORLD, &myRank);

    if (myRank == 0) {
        answer = 42;
    }

    printf("BEFORE the broadcast, process %d's answer = %d\n",
           myRank, answer);

    MPI_Bcast(&answer, 1, MPI_INT, 0, MPI_COMM_WORLD);

    printf("AFTER the broadcast, process %d's answer = %d\n",
           myRank, answer);

    MPI_Finalize();
    return 0;
}
```

Experiment 8 – MPI Scatter and Gather

Demonstration of MPI_Scatter() and MPI_Gather().

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    int numProcs, rank;

    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &numProcs);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int masterData[4];  // Array used only by master process
    int myData;         // Each process has its own local data
    int i;

    if (rank == 0) {
        for (i = 0; i < numProcs; i++)
            masterData[i] = i;

        printf("Step 1: Master (rank %d) initialized the data:\n", rank);

        for (i = 0; i < numProcs; i++)
            printf(" masterData[%d] = %d\n", i, masterData[i]);

        printf("\n");
    }

    // Step 2: Scatter data
    MPI_Scatter(masterData, 1, MPI_INT,
                &myData, 1, MPI_INT, 0, MPI_COMM_WORLD);

    printf("Step 2: Process %d received value %d from master\n",
           rank, myData);

    // Step 3: Each process updates its local value
    myData = 5;

    printf("Step 3: Process %d updated its local value to %d\n",
           rank, myData);

    // Step 4: Gather all modified data back to master
    MPI_Gather(&myData, 1, MPI_INT,
               masterData, 1, MPI_INT, 0, MPI_COMM_WORLD);

    // Step 5: Master displays gathered results
    if (rank == 0) {
        printf("\nStep 4: Master (rank %d) gathered updated values:\n",
               rank);

        for (i = 0; i < numProcs; i++)
            printf(" masterData[%d] = %d\n", i, masterData[i]);

        printf("\n");
    }

    MPI_Finalize();
    return 0;
}
```

Experiment 9 – MPI Reduce and Allreduce

Demonstration of MPI_Reduce() and MPI_Allreduce() using SUM, PRODUCT, MAX and MIN.

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv)
{
    int rank, size;
    int value, sum, product, max, min;
    int all_sum, all_product, all_max, all_min;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    value = rank + 1;  // Each process has a unique value

    // ---- MPI_Reduce ----
    MPI_Reduce(&value, &sum, 1, MPI_INT,
               MPI_SUM, 0, MPI_COMM_WORLD);

    MPI_Reduce(&value, &product, 1, MPI_INT,
               MPI_PROD, 0, MPI_COMM_WORLD);

    MPI_Reduce(&value, &max, 1, MPI_INT,
               MPI_MAX, 0, MPI_COMM_WORLD);

    MPI_Reduce(&value, &min, 1, MPI_INT,
               MPI_MIN, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("=== MPI_Reduce results at root ===\n");
        printf("SUM = %d\n", sum);
        printf("PRODUCT = %d\n", product);
        printf("MAX = %d\n", max);
        printf("MIN = %d\n", min);
    }

    // ---- MPI_Allreduce ----
    MPI_Allreduce(&value, &all_sum, 1, MPI_INT,
                  MPI_SUM, MPI_COMM_WORLD);

    MPI_Allreduce(&value, &all_product, 1, MPI_INT,
                  MPI_PROD, MPI_COMM_WORLD);

    MPI_Allreduce(&value, &all_max, 1, MPI_INT,
                  MPI_MAX, MPI_COMM_WORLD);

    MPI_Allreduce(&value, &all_min, 1, MPI_INT,
                  MPI_MIN, MPI_COMM_WORLD);

    printf("Process %d: ALL_SUM = %d, ALL_PROD = %d, "
           "ALL_MAX = %d, ALL_MIN = %d\n",
           rank, all_sum, all_product, all_max, all_min);
```

```
    MPI_Finalize();
    return 0;
}
```